Phase 4: Graph Knowledge Base

Overview

The Graph Knowledge Base adds a semantic layer on top of the existing vector-based RAG system. Using Neo4j Community Edition, it builds an entity graph from ingested documents by extracting named entities (people, organizations, products, processes, etc.) and their relationships.

This graph enables:

- Structured queries about relationships between concepts
- Entity disambiguation across documents
- Contextual retrieval that combines vector similarity with graph traversal
- Visual exploration of knowledge structure

Architecture

Key design decision: Graph extraction is non-blocking. If Neo4j is unavailable or LLM extraction fails, documents remain fully available via vector search. The graph is supplementary.

How Entity Extraction Works

- After a document is chunked and stored in Qdrant, the ingestion worker queues a graph extraction job
- The graph extraction worker processes chunks in batches of 5 (concurrent LLM calls)
- Each chunk is sent to the cheapest LLM tier (fast-cheap/DeepSeek) with a structured extraction prompt
- The LLM returns JSON with entities and relationships
- Results are validated, filtered by confidence threshold, and deduplicated across chunks
- Unique entities are stored as Neo4j nodes; relationships as edges

Entity Types

Core (business documents):

PERSON, ORGANIZATION, PRODUCT, SERVICE, PROCESS, DEPARTMENT, ROLE, LOCATION, DOCUMENT, REGULATION, TECHNOLOGY, EVENT, METRIC, CONCEPT

Extended (professional knowledge & methodology):

PRINCIPLE, PATTERN, METHODOLOGY, SKILL, CERTIFICATION, FRAMEWORK, ANTI_PATTERN, DECISION, TRADEOFF, TOOL, PLATFORM, INDUSTRY, USE_CASE, RISK, MITIGATION

Relationship Types

Core:

WORKS_FOR, MANAGES, REPORTS_TO, PRODUCES, CONSUMES, DEPENDS_ON, PART_OF, CONTAINS, RELATED_TO, RESPONSIBLE_FOR, LOCATED_IN, USES, PROVIDES, REQUIRES, PRECEDES, FOLLOWS, TRIGGERS

Extended (knowledge & methodology):

APPLIES, SOLVES, CONTRADICTS, ENABLES, MITIGATES, REPLACES, EVOLVED_FROM, VALIDATED_BY, RECOMMENDED_FOR, INCOMPATIBLE_WITH, COMPOSED_OF, INSTANTIATES, SPECIALIZES, COMPLEMENTS

Professional Knowledge Graph

Purpose

Beyond client business documents, the graph models the consultant's professional knowledge — the methodologies, principles, patterns, and experience that inform advisory work. This transforms the platform from a document chatbot into an opinionated advisor that can reason about trade-offs.

Knowledge Domains Modeled

The consultant's experience spans multiple domains. Each is ingested as structured knowledge into the graph:

Domain | Entity Types Used | Example Entities

Software Architecture | PRINCIPLE, PATTERN, ANTI_PATTERN, TRADEOFF | "SOLID", "Circuit Breaker", "Distributed Monolith", "Latency vs Consistency"

AI Engineering | METHODOLOGY, TOOL, RISK, MITIGATION | "RAG Pipeline", "Prompt Engineering", "Hallucination", "Golden Dataset Eval"

Agile & DevOps | FRAMEWORK, METHODOLOGY, METRIC | "Scrum", "Trunk-Based Dev", "DORA Metrics", "Blameless Postmortem"

Tech Leadership | SKILL, PATTERN, FRAMEWORK | "Delegation Matrix", "1:1 Coaching", "ADR Writing", "Team Topologies"

Digital Transformation | PROCESS, METHODOLOGY, RISK | "As-Is/To-Be Mapping", "Change Management", "Shadow IT"

Security & Compliance | REGULATION, PRINCIPLE, RISK | "OWASP Top 10", "GDPR", "Zero Trust", "Least Privilege"

Resilience & Operations | PATTERN, METRIC, TOOL | "Circuit Breaker", "Golden Signals", "Idempotency", "Chaos Engineering"

Wellbeing & Sustainability | CONCEPT, PATTERN, ANTI_PATTERN | "Sustainable Pace", "Cognitive Load", "Burnout Signals", "Hero Culture"

How It Works in Practice

When a client asks a question, the hybrid RAG system:

- Vector search (Qdrant) → finds relevant document chunks
- Graph traversal (Neo4j) → finds related principles, patterns, and trade-offs
- Fusion → combines both contexts into the LLM prompt

Example flow:

```
Client question: "Dovremmo passare a microservizi?"

Vector search → finds client's architecture docs, current monolith issues
Graph traversal → finds:
  - PRINCIPLE "Bounded Context" --ENABLES--> PATTERN "Microservices"
  - ANTI_PATTERN "Distributed Monolith" --CONTRADICTS--> PATTERN "Microservices"
  - PRINCIPLE "Conway's Law" --APPLIES--> DECISION "Team Structure Before Architecture"
  - TRADEOFF "Operational Complexity vs Independent Deployability"
  - METHODOLOGY "Strangler Fig" --SOLVES--> PROCESS "Monolith Decomposition"

Agent response: informed opinion with trade-offs, not just document retrieval
```

Ingestion of Professional Knowledge

Professional knowledge is ingested from the consultant's documentation (tracks/study/ files, playbooks, manuals). The extraction prompt is tuned to identify:

- Principles: immutable rules ("Every remote call will fail", "Coverage != Confidence")
- Patterns: reusable solutions (Circuit Breaker, Result Pattern, ADR)
- Anti-patterns: things to avoid (God Class, Shared DB between services)
- Trade-offs: named tensions (Consistency vs Availability, Speed vs Safety)
- Methodologies: structured approaches (DDD, TDD London, Strangler Fig)
- Decisions: recorded choices with context (ADR format)

Graph Schema for Professional Knowledge

```
// Principles
(:Entity:PRINCIPLE {name: "Idempotency", domain: "resilience",
  description: "Every critical operation subject to retry must be idempotent"})

// Patterns with context
(:Entity:PATTERN {name: "Circuit Breaker", domain: "resilience",
  when: "External service calls with unpredictable latency",
  tradeoff: "Added complexity vs fault isolation"})

// Anti-patterns with consequences
(:Entity:ANTI_PATTERN {name: "Distributed Monolith", domain: "architecture",
  signal: "Every service must know too much about others",
  consequence: "Distributed the problem, not the solution"})

// Trade-offs as first-class entities
(:Entity:TRADEOFF {name: "Monolith vs Microservices", domain: "architecture",
  left: "Simplicity, speed, low ops cost",
  right: "Independent deploy, team autonomy, scaling"})

// Relationships encode reasoning chains
(:PRINCIPLE {name: "Bounded Context"})-[:ENABLES]->(:PATTERN {name: "Microservices"})
(:PATTERN {name: "Microservices"})-[:REQUIRES]->(:PRINCIPLE {name: "Independent Data Stores"})
(:ANTI_PATTERN {name: "Shared DB"})-[:CONTRADICTS]->(:PRINCIPLE {name: "Loose Coupling"})
(:METHODOLOGY {name: "Strangler Fig"})-[:SOLVES]->(:PROCESS {name: "Monolith Migration"})
(:PATTERN {name: "Modular Monolith"})-[:REPLACES]->(:PATTERN {name: "Microservices"})
  // when: team < 20 people, single deploy unit acceptable
```

Confidence and Validation

Professional knowledge entities have additional metadata:

Field	Purpose
confidence	0.0-1.0 — how certain the extraction is

source | Which document/manual it came from
domain | Which consulting track it belongs to
validated | Boolean — manually confirmed by consultant
applicability | Conditions under which this knowledge applies

The consultant can validate/correct entities via the Graph Explorer in the dashboard, building a curated knowledge graph over time.

Pre-seeded Knowledge

On platform setup, a base graph is pre-seeded from the consultant's study materials:

- 9 STUDY-*.md files → ~200-400 entities per file
- Playbooks → operational methodology entities
- Architecture guidelines → principle/pattern entities

This gives the platform immediate advisory capability without waiting for client documents.

Consultant Knowledge Seed — Modalita e Principi

The following represents the consultant's core operating principles and methodologies, modeled as graph entities. These are pre-seeded on first platform setup and form the "personality" of the AI advisor.

Architectural Principles (seeded)

Entity	Type	Key Relationship
"Architecture = Explicit Trade-offs"	PRINCIPLE	CONTRADICTS → "Architecture as Dogma"
"Fitness Functions over Slides"	PRINCIPLE	VALIDATES → any DECISION
"ADR Mandatory"	PRINCIPLE	ENABLES → "Architectural Memory"
"Design for Change ≠ Over-engineering"	PRINCIPLE	MITIGATES → "YAGNI Violation"
"Modular Monolith is Serious"	PATTERN	REPLACES → "Premature Microservices"
"Bounded Context First"	PRINCIPLE	PRECEDES → "Service Extraction"
"Core Domain gets Max Quality"	PRINCIPLE	APPLIES → "Resource Allocation"
"Anticorruption Layer on External"	PATTERN	MITIGATES → "Model Contamination"
"Conway's Law"	PRINCIPLE	REQUIRES → "Org Change Before Arch Change"
"No Microservice Without Proven Boundary"	PRINCIPLE	CONTRADICTS → "Technology-Driven Decomposition"

Testing Principles (seeded)

Entity	Type	Key Relationship
"Coverage ≠ Confidence"	PRINCIPLE	CONTRADICTS → "Coverage Target as Goal"
"Test Behavior Not Implementation"	PRINCIPLE	MITIGATES → "Brittle Tests"
"Integration > Superficial Unit"	TRADEOFF	APPLIES → "Business-Critical Systems"
"Contract Tests Mandatory"	PRINCIPLE	ENABLES → "Independent Release"
"E2E Few, Stable, Business-Critical"	PRINCIPLE	COMPLEMENTS → "Test Pyramid"
"Testability as Architectural Property"	PRINCIPLE	ENABLES → "Correct Placement"
"Fast Feedback as Advantage"	PRINCIPLE	CONTRADICTS → "Slow Pipeline"
"Deploy ≠ Release"	PRINCIPLE	ENABLES → "Feature Flags"

Resilience & Operations (seeded)

Entity	Type	Key Relationship
"Every Remote Call Will Fail"	PRINCIPLE	REQUIRES → "Circuit Breaker"
"Idempotency is a Lifesaver"	PRINCIPLE	ENABLES → "Safe Retry"

"Observability = Distributed Debugger" | PRINCIPLE | REQUIRES → "Structured Logging + Traces + Metrics"
"Golden Signals" | FRAMEWORK | COMPOSED_OF → "Latency, Rate, Errors, Saturation"
"Timeout + Retry + Breaker" | PATTERN | MITIGATES → "Cascade Failure"

AI Engineering (seeded)

Entity | Type | Key Relationship
"AI as System Not Demo" | PRINCIPLE | REQUIRES → "Production Guardrails"
"Evaluation Before Enthusiasm" | PRINCIPLE | PRECEDES → "Production Deploy"
"RAG: Retrieval Before Generation" | PRINCIPLE | ENABLES → "Precise Context"
"Multi-Agent: Powerful but Fragile" | TRADEOFF | MITIGATES → "Cost Routing"
"Cost Routing and Model Selection" | PATTERN | ENABLES → "90% cases at 1/10 cost"
"Security by Design for AI" | PRINCIPLE | MITIGATES → "Prompt Injection, Data Leakage"
"Golden Dataset Required" | PRINCIPLE | ENABLES → "Reliable Evaluation"

Team & Ownership (seeded)

Entity | Type | Key Relationship
"Cognitive Load Sustainability" | PRINCIPLE | MITIGATES → "Burnout"
"Product Engineering Ownership" | PRINCIPLE | ENABLES → "End-to-End Responsibility"
"Leadership Without Bureaucracy" | PRINCIPLE | ENABLES → "Team Autonomy"
"Sustainable Pace" | PATTERN | CONTRADICTS → "Hero Culture"
"Psychological Safety" | PRINCIPLE | ENABLES → "Innovation"

Operating Methodology (seeded)

Entity | Type | Key Relationship
"Challenge Before Analyze" | METHODOLOGY | PRECEDES → "Solution Design"
"Epistemic Integrity" | PRINCIPLE | REQUIRES → "Confidence Markers"
"5 Whys to Root Cause" | METHODOLOGY | ENABLES → "Correct Problem Identification"
"Evidence-Based Decisions" | PRINCIPLE | CONTRADICTS → "Opinion-Based Architecture"
"Name the Risk Before the Conclusion" | PRINCIPLE | MITIGATES → "Silent Failure"
"Ownership of Every Decision" | PRINCIPLE | ENABLES → "Accountability"
"Ask Before Assume" | PRINCIPLE | MITIGATES → "Implicit Assumptions"

How Seeding Works

```
// On first tenant setup, or when consultant updates their knowledge base:
await seedProfessionalKnowledge(tenantId, {
  source: 'consultant-principles',
  documents: [
    'tracks/study/STUDY-ARCH-Scaling.md',
    'tracks/study/STUDY-AI-Adoption.md',
    'tracks/study/STUDY-AIA-Piattaforma.md',
    'tracks/study/STUDY-AGILE-DevOps.md',
    'tracks/study/STUDY-FCTO-FractionalCTO.md',
    'tracks/study/STUDY-LEAD-Leadership.md',
    'tracks/study/STUDY-WELL-Wellbeing.md',
    'tracks/study/STUDY-DIGI-Trasformazione.md',
    'tracks/study/STUDY-ZERO-DigitalStarter.md',
  ],
  extractionMode: 'professional-knowledge', // special prompt template
  autoValidate: true, // consultant's own material = pre-validated
});
```

The professional-knowledge extraction mode uses a specialized prompt that looks for:

- Normative statements ("must", "never", "always", "required")
- Conditional rules ("when X, do Y", "only if", "unless")
- Trade-off formulations ("X vs Y", "at the cost of", "in exchange for")
- Anti-pattern signals ("avoid", "don't", "common mistake", "failure mode")
- Methodology steps ("first... then...", "before X, ensure Y")

This differs from the standard business-document mode which extracts facts, entities, and their relationships without normative framing.

Querying the Graph

API Endpoints

Method	Path	Description
GET	/api/graph/entities	List entities (paginated, filterable by type)
GET	/api/graph/entities/:id	Get entity with relationships
GET	/api/graph/entities/:id/context	Get entity + 1-hop neighbors
GET	/api/graph/search?query=...	Full-text search on entity names
GET	/api/graph/subgraph?ids=...	Get subgraph for specified entity IDs
GET	/api/graph/stats	Graph statistics (entity counts, top entities)
GET	/api/graph/documents/:docId/entities	Entities from a specific document
POST	/api/graph/entities/:id/merge	Merge duplicate entities
DELETE	/api/graph/entities/:id	Remove entity and its relationships

Search Examples

```
# Search for entities matching "pricing"
curl -H "Authorization: Bearer $TOKEN" \
  "http://localhost:3000/api/graph/search?query=pricing&limit=10"

# Get entity context (1-hop neighborhood)
curl -H "Authorization: Bearer $TOKEN" \
  "http://localhost:3000/api/graph/entities/abc123/context"

# Filter by type
curl -H "Authorization: Bearer $TOKEN" \
  "http://localhost:3000/api/graph/search?query=john&types=PERSON"
```

Merging Duplicates

The extraction process may produce duplicate entities across documents (e.g., "Microsoft Corp" and "Microsoft Corporation"). Use the merge endpoint:

```
# Merge sourceId into targetId (all relationships transfer to target)
curl -X POST -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"targetEntityId": "target-id-here"}' \
  "http://localhost:3000/api/graph/entities/source-id-here/merge"
```

The merge operation:

- Transfers all incoming relationships from source to target
- Transfers all outgoing relationships from source to target
- Deletes the source entity

Performance Considerations

LLM Calls

- Entity extraction uses the cheapest model tier (fast-cheap)
- Each chunk requires 1 LLM call (~500-2000 tokens per chunk)
- A 10-page document (~20 chunks) costs approximately 20 LLM calls
- Graph worker has low concurrency (2) and rate limiting (5 jobs/min)

Neo4j

- Indexes on tenantId, type, sourceDocumentId for fast filtering
- Full-text index on entity names for search
- All queries are tenant-scoped (multi-tenancy enforced at query level)
- Graph traversal limited to configurable depth (default 2 hops)

Memory

- Neo4j heap: 1GB max (configurable via docker-compose)
- Page cache: 512MB
- Suitable for up to ~1M entities per instance

Cost Considerations

Document Size	Chunks	LLM Calls	Estimated Cost (DeepSeek)
1 page	2-3	2-3	~$0.001
10 pages	15-25	15-25	~$0.01
50 pages	75-125	75-125	~$0.05

100 pages | 150-250 | 150-250 | ~$0.10

Graph extraction can be disabled per-tenant or globally via GRAPH_EXTRACTION_ENABLED=false.

Configuration

```
# Neo4j connection
NEO4J_URL=bolt://neo4j:7687
NEO4J_USER=neo4j
NEO4J_PASSWORD=neo4j_dev_password

# Extraction settings
GRAPH_EXTRACTION_ENABLED=true
GRAPH_EXTRACTION_MODEL=fast-cheap
GRAPH_EXTRACTION_MIN_CONFIDENCE=0.3
```

Multi-Tenancy

- All Neo4j nodes have a tenantId property
- All queries filter by tenantId (enforced in repository layer)
- Composite indexes include tenantId for performance
- No cross-tenant data leakage is possible through the API

Development

```
# Start infrastructure with Neo4j
docker compose -f docker-compose.yml -f docker-compose.dev.yml up -d

# Access Neo4j Browser (dev only)
open http://localhost:7474

# Connect with: neo4j / neo4j_dev_password
# Database: neo4j

# Example Cypher queries:
# Count all entities for a tenant
MATCH (e:Entity {tenantId: "your-tenant-id"}) RETURN count(e);

# Find all PERSON entities
MATCH (e:Entity:PERSON {tenantId: "your-tenant-id"}) RETURN e.name, e.confidence;

# Show entity relationships
MATCH (e:Entity {tenantId: "your-tenant-id"})-[r:GRAPH_REL]->(other)
RETURN e.name, r.type, other.name LIMIT 50;
```